

Advanced Probabilistic Machine Learning

Ralf Herbrich, Rainer Schlosser

Approximate Inference: Expectation Propagation

Overview

1. Distance Measures for Distributions
2. Approximating Marginals: Expectation Propagation
3. Approximating Normalization Constants
4. Application: Bayesian Ranking

1. **Distance Measures for Distributions**
2. Approximating Marginals: Expectation Propagation
3. Approximating Normalization Constants
4. Application: Bayesian Ranking

Distance Measures: α -Divergence

- **Problem.** We have a non-Gaussian normalized marginal $p(\cdot)$ and would like to approximate it by a Gaussian $q(\cdot) = \mathcal{N}(\cdot; \mu, \sigma^2)$. What is the optimal approximation?
- **Solution.** To define “optimality”, we need a distance measure between probability densities $p(\cdot)$ and $q(\cdot)$!
- **α -Divergence (Amari, 1985).** Given two probability densities $p(\cdot)$ and $q(\cdot)$ and $\alpha \in \mathbb{R} \setminus \{0,1\}$ the α -divergence $D_\alpha[p, q]$ is defined by

$$D_\alpha[p, q] = \frac{1}{\alpha(1-\alpha)} \cdot \left(1 - \int_{-\infty}^{+\infty} \left[\frac{p(x)}{q(x)} \right]^\alpha \cdot q(x) \, dx \right)$$

Expectation of $\left[\frac{p(x)}{q(x)} \right]^\alpha$ over $q(x)$

If $p = q$ then $\left[\frac{p(x)}{q(x)} \right]^\alpha = 1$
and the expectation is 1

- **Non-Negativity:** If $p = q$ then $D_\alpha[p, q] = 0$; otherwise $D_\alpha[p, q] > 0$
- **Asymmetry:** $D_\alpha[p, q] \neq D_\alpha[q, p]$
- **Flexibility:**
 - $\alpha > 1$ gives more weight to regions where $p(x) > q(x)$
 - $\alpha < 1$ gives more weight to regions where $q(x) > p(x)$

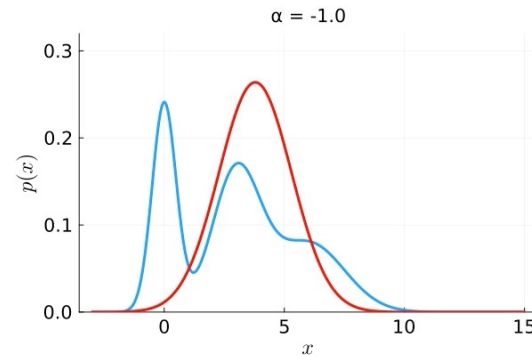
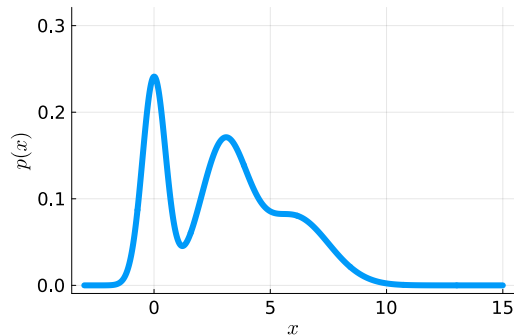


Shun'ichi Amari (甘利 俊)
(1936)

**Advanced Probabilistic
Machine Learning**

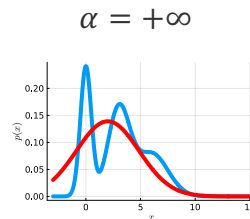
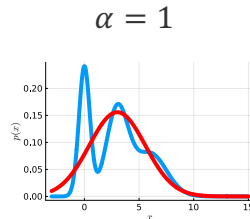
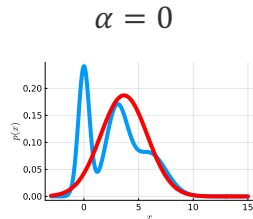
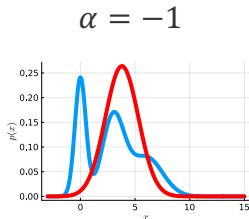
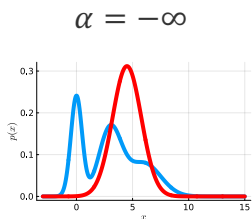
Unit 5 – Approximate Inference:
Expectation Propagation

α -Divergence with a Gaussian in Pictures



zero-forcing & mode seeking

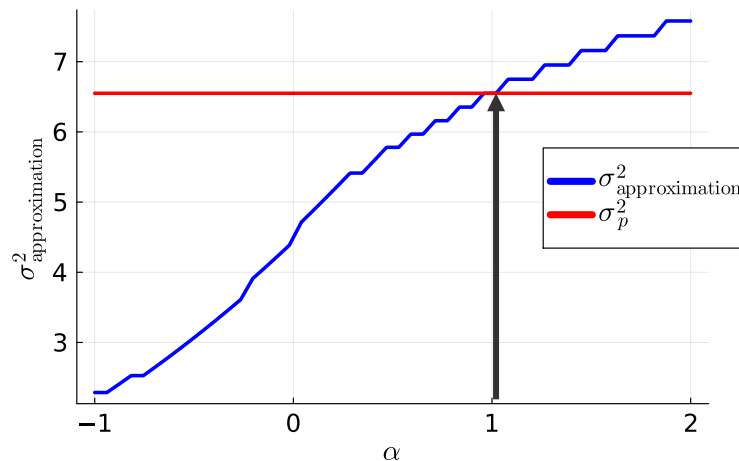
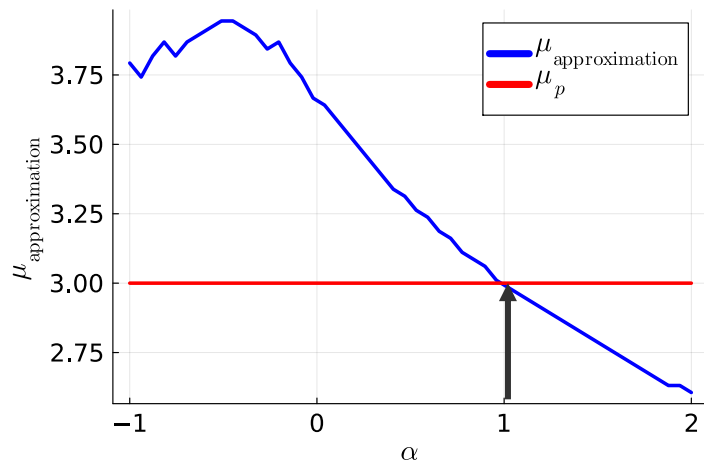
inclusive & support seeking



**Advanced Probabilistic
Machine Learning**

*Unit 5 – Approximate Inference:
Expectation Propagation*

α -Divergence with a Gaussian and Moment Matching



- Only for $\alpha = 1$ both the first and second moment (that is, mean and variance) are matched with that of the approximation!
- The case $\alpha = 1$ is a limit case.

$\alpha = 1$: KL Divergence

- **Theorem (Limit $\alpha \rightarrow 1$).** Given two probability densities $p(\cdot)$ and $q(\cdot)$ the limit of the α -divergence $D_\alpha[p, q]$ for $\alpha \rightarrow 1$ is the Kullback-Leibler divergence

$$\lim_{\alpha \rightarrow 1} D_\alpha[p, q] = \text{KL}[p, q] := \int_{-\infty}^{+\infty} \log\left(\frac{p(x)}{q(x)}\right) \cdot p(x) \, dx$$

- **Proof:** Taking limits, we have

$$\begin{aligned} \lim_{\alpha \rightarrow 1} D_\alpha[p, q] &= \lim_{\alpha \rightarrow 1} \frac{1}{\alpha(1-\alpha)} \cdot \left(1 - \int_{-\infty}^{+\infty} \left[\frac{p(x)}{q(x)} \right]^\alpha \cdot q(x) \, dx \right) \quad \text{L'Hôpital's rule!} \\ &= \lim_{\alpha \rightarrow 1} \frac{1}{1-2\alpha} \cdot \left(- \int_{-\infty}^{+\infty} \log\left(\frac{p(x)}{q(x)}\right) \cdot \left[\frac{p(x)}{q(x)} \right]^\alpha q(x) \, dx \right) \quad \text{Note that } \frac{d}{dz} b^z = \log(b) \cdot b^z \\ &= \int_{-\infty}^{+\infty} \log\left(\frac{p(x)}{q(x)}\right) \cdot p(x) \, dx \end{aligned}$$



Solomon Kullback
(1909 – 1994)



Richard Leibler
(1914 – 2003)

**Advanced Probabilistic
Machine Learning**

Unit 5 – Approximate Inference:
Expectation Propagation

Minimizing the KL Divergence

- **Theorem.** Given any distribution $p(\cdot)$ over a random variable X and an exponential family distribution over X characterized by the sufficient statistic $\mathbf{T}(\cdot)$, the global minimizer of the KL-divergence $\text{KL} \left[p(\cdot), \exp \left(\boldsymbol{\theta}^T \mathbf{T}(\cdot) - A(\boldsymbol{\theta}) \right) \right]$ is given by $\boldsymbol{\theta}^*$ such that

$$E_{X \sim \exp(\boldsymbol{\theta}^T \mathbf{T}(\cdot) - A(\boldsymbol{\theta}))}[\mathbf{T}(X)] = E_{X \sim p(\cdot)}[\mathbf{T}(X)]$$

- Intuitively, we find the minimizer by matching the expected sufficient statistic $E[\mathbf{T}(X)]$ expressed as functions of the natural parameters $\boldsymbol{\theta}$ and solving for $\boldsymbol{\theta}$.
- **Corollary (Moment Matching).** Given any distribution $p(\cdot)$ the minimizer $\boldsymbol{\mu}^*, \boldsymbol{\Sigma}^*$ of the KL divergence $\text{KL}[p(\cdot), \mathcal{N}(\cdot; \boldsymbol{\mu}, \boldsymbol{\Sigma})]$ to a multivariate Gaussian distribution has

$$\boldsymbol{\mu}^* = E_{X \sim p(\cdot)}[\mathbf{X}] \quad \text{and} \quad \boldsymbol{\Sigma}^* = E_{X \sim p(\cdot)}[(\mathbf{X} - \boldsymbol{\mu}^*)(\mathbf{X} - \boldsymbol{\mu}^*)^T]$$

- **Proof.** Noting that the expected sufficient statistic is $E_{X \sim p(\cdot)}[\mathbf{X}]$ and $E_{X \sim p(\cdot)}[\mathbf{X}\mathbf{X}^T]$ we see

$$\boldsymbol{\mu}^* = E_{X \sim p(\cdot)}[\mathbf{X}] \quad \boldsymbol{\Sigma}^* = E_{X \sim p(\cdot)}[\mathbf{X}\mathbf{X}^T] - (E_{X \sim p(\cdot)}[\mathbf{X}])(E_{X \sim p(\cdot)}[\mathbf{X}])^T$$

Proof of KL Divergence Minimizer

- Let us write the KL divergence as a function of the natural parameters θ

$$\begin{aligned} \text{KL} [p(\cdot), \exp(\theta^T \mathbf{T}(\cdot) - A(\theta))] &= \int p(x) \cdot \log \left[\frac{p(x)}{\exp(\theta^T \mathbf{T}(x) - A(\theta))} \right] dx \\ &= \int p(x) \cdot \log[p(x)] dx - \theta^T E_{X \sim p(\cdot)}[\mathbf{T}(x)] - A(\theta) \end{aligned}$$

Linearity of expectation

Independent of x

- Taking the derivative of the KL divergence w.r.t. θ and setting this to 0

$$\frac{\partial}{\partial \theta} A(\theta^*) = E_{X \sim p(\cdot)}[\mathbf{T}(x)]$$

- Taking the derivative of the log-normalization constant

$$\begin{aligned} \frac{\partial}{\partial \theta} A(\theta^*) &= \frac{\partial}{\partial \theta} \log \left[\int \exp(\theta^T \mathbf{T}(x)) dx \right] = \frac{1}{\int \exp(\theta^T \mathbf{T}(x)) dx} \cdot \int \mathbf{T}(x) \cdot \exp(\theta^T \mathbf{T}(x)) dx \\ &= \int \mathbf{T}(x) \cdot \frac{\exp(\theta^T \mathbf{T}(x))}{\int \exp(\theta^T \mathbf{T}(\tilde{x})) d\tilde{x}} dx = E_{X \sim \exp(\theta^T \mathbf{T}(\cdot) - A(\theta))}[\mathbf{T}(X)] \end{aligned}$$

**Advanced Probabilistic
Machine Learning**

Unit 5 – Approximate Inference:
Expectation Propagation

Overview

1. Distance Measures for Distributions
- 2. Approximating Marginals: Expectation Propagation**
3. Approximating Normalization Constants
4. Application: Bayesian Ranking

**Advanced Probabilistic
Machine Learning**

*Unit 5 – Approximate Inference:
Expectation Propagation*

Sum-Product Algorithm Revisited

- The key operation for factor $f(x_1, x_2, \dots, x_n)$ and variable X_1 is

$$m_{f \rightarrow X_1}(x_1) = \sum_{\{x_2\}} \cdots \sum_{\{x_n\}} f(x_1, x_2, \dots, x_n) \prod_{j=2}^n m_{X_j \rightarrow f}(x_j)$$

If all $m_{X_j \rightarrow f}(x_j)$ are Gaussian, the result **might not be** Gaussian!

- Based on outgoing messages, we can compute both non-normalized marginals $p_X(\cdot)$ and $m_{X \rightarrow f}(\cdot)$

$$p_X(x) = \prod_{f \in \text{ne}(X)} m_{f \rightarrow X}(x) \quad m_{X \rightarrow f}(x) = \frac{p_X(x)}{m_{f \rightarrow X}(x)}$$

If all $m_{X_j \rightarrow f}(x_j)$ are Gaussian, the result **must be** Gaussian!

Idea:

- We approximate all outgoing messages $m_{f \rightarrow X}(\cdot)$ by a Gaussian $\hat{m}_{f \rightarrow X}(\cdot) = \mathcal{N}(\cdot; \mu, \sigma^2)$
- We measure the approximation quality in the normalized marginal, **not** the outgoing message

Advanced Probabilistic Machine Learning

Unit 5 – Approximate Inference: Expectation Propagation

$$\hat{p}(\cdot) = \arg \min_{\mu, \sigma^2} \text{KL} \left[\frac{m_{f \rightarrow X}(\cdot) \cdot \hat{m}_{X \rightarrow f}(\cdot)}{\int_{-\infty}^{+\infty} m_{f \rightarrow X}(\tilde{x}) \cdot \hat{m}_{X \rightarrow f}(\tilde{x}) d\tilde{x}}, \frac{\mathcal{N}(\cdot; \mu, \sigma^2) \cdot \hat{m}_{X \rightarrow f}(\cdot)}{\int_{-\infty}^{+\infty} \mathcal{N}(\tilde{x}; \mu, \sigma^2) \cdot \hat{m}_{X \rightarrow f}(\tilde{x}) d\tilde{x}} \right]$$

True normalized marginal with approximate incoming message

Approximate marginal with approximate incoming message

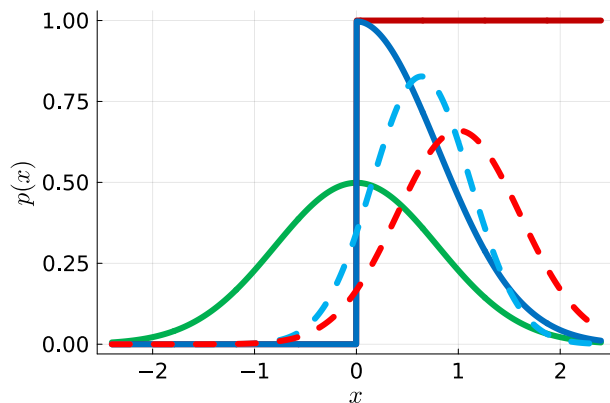
Approximate Message Passing: Example

$$f(x) = \mathbb{I}(x > 0)$$

$$\hat{m}_{X \rightarrow f}(x) \propto \frac{\hat{p}_X(x)}{\hat{m}_{f \rightarrow X}(x)} \longrightarrow p_X(x) \propto f(x) \cdot \hat{m}_{X \rightarrow f}(x)$$

$$\hat{p}_X(x) = \mathcal{N}(x; E_{X \sim p_X}[X], \text{var}_{X \sim p_X}[X])$$

$$\hat{m}_{f \rightarrow X}(x) \propto \frac{\hat{p}_X(x)}{\hat{m}_{X \rightarrow f}(x)}$$

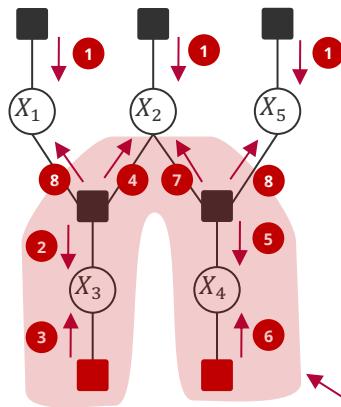


Expectation Propagation

- **Idea:** If we have factors in the factor graph that require approximate messages, we keep iterating on the whole path between them until convergence minimizing $KL(p(\cdot) | \mathcal{N}(\cdot; \mu, \sigma^2))$ locally for the affected marginals of the approximate factor.
- **Theorem (Minka, 2003):** *The approximate message passing algorithm using the Kullback-Leibler convergence will always converge if the approximating distribution is in the exponential family!*



Tom Minka



Advanced Probabilistic Machine Learning

Unit 5 – Approximate Inference:
Expectation Propagation

- $$p_X(x) \propto \mathbb{I}(l < x < u) \cdot \mathcal{N}(x; \mu, \sigma^2)$$

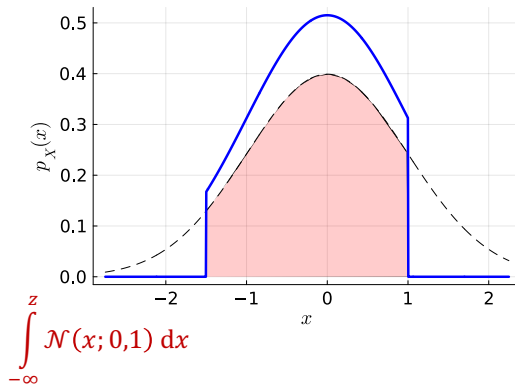
- $$E[X^0] = \Phi(t_{u-\mu}) - \Phi(t_{l-\mu})$$

$$E[X^1] = \mu + \sigma \cdot \frac{\mathcal{N}(t_{l-\mu}) - \mathcal{N}(t_{u-\mu})}{\Phi(t_{u-\mu}) - \Phi(t_{l-\mu})}$$

Additive correction that goes to zero
as $u \rightarrow \infty$ and $l \rightarrow -\infty$

$$E[X^2] = \mu^2 + \sigma^2 \cdot \left[1 - \frac{t_{u+\mu} \cdot \mathcal{N}(t_{u-\mu}) - t_{l+\mu} \cdot \mathcal{N}(t_{l-\mu})}{\Phi(t_{u-\mu}) - \Phi(t_{l-\mu})} \right]$$

Multiplicative correction that goes to one
as $u \rightarrow \infty$ and $l \rightarrow -\infty$



Doubly-Truncated Gaussians (ctd)

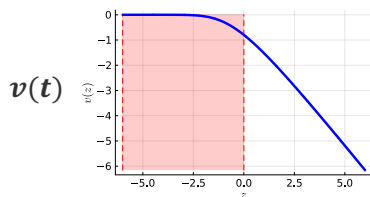
- Using the variance decomposition theorem we see

$$E[X] = \mu + \sigma \cdot v_{l,u}\left(\frac{\mu}{\sigma}\right) \quad \text{var}[X] = \sigma^2 \cdot \left[1 - w_{l,u}\left(\frac{\mu}{\sigma}\right)\right]$$

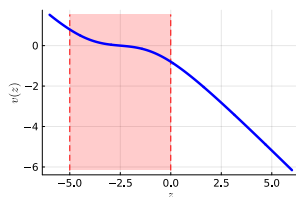
$$v_{l,u}(z) := \frac{\mathcal{N}(l-z) - \mathcal{N}(u-z)}{\Phi(u-z) - \Phi(l-z)}$$

$$w_{l,u}(z) := \frac{(u+z) \cdot \mathcal{N}(u-z) - (l+z) \cdot \mathcal{N}(l-z)}{\Phi(u-z) - \Phi(l-z)} + v_{l,u}(z) \cdot [2z + v_{l,u}(z)]$$

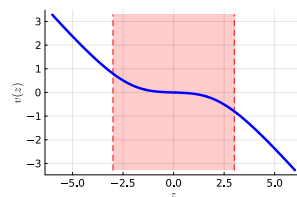
$x < 0$



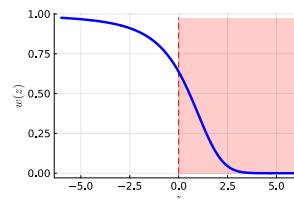
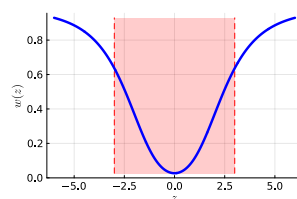
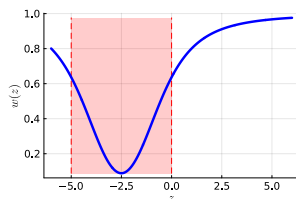
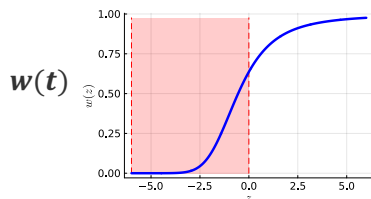
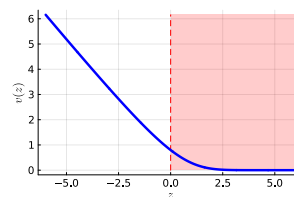
$-5 \leq x < 0$



$-3 \leq x < 3$



$0 \leq x$



**Advanced Probabilistic
Machine Learning**

*Unit 5 – Approximate Inference:
Expectation Propagation*

Overview

1. Distance Measures for Distributions
2. Approximating Marginals: Expectation Propagation
- 3. Approximating Normalization Constants**
4. Application: Bayesian Ranking

**Advanced Probabilistic
Machine Learning**

*Unit 5 – Approximate Inference:
Expectation Propagation*

Normalization Constant

- **Normalization Constant:** Given a factor graph with factors $f_1, \dots, f_m$, each over a subset of n variables $X_1, X_2, \dots, X_n$, the normalization constant Z is defined as the sum over all variables

$$Z = \sum_{\{x_1\}} \cdots \sum_{\{x_n\}} f_1(\mathbf{x}_{\text{ne}(f_1)}) \cdot f_2(\mathbf{x}_{\text{ne}(f_2)}) \cdots f_m(\mathbf{x}_{\text{ne}(f_m)})$$

- **Inference in Factor Graphs:** In order to learn from data D we

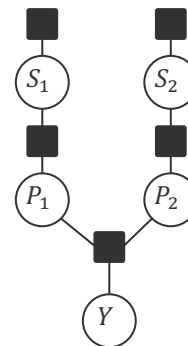
1. **Modelling:** Formulate a joint model $p(\boldsymbol{\theta}, D)$ of parameters $\boldsymbol{\theta} = \theta_1, \dots, \theta_n$ and data D

$$Z = \sum_{\{\theta_1\}} \cdots \sum_{\{\theta_n\}} \sum_{\{D\}} p(\boldsymbol{\theta}, D) = 1$$

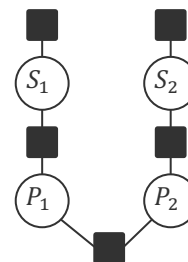
2. **Conditioning:** Remove the variables that represent data D from the factor graph

$$Z = \sum_{\{\theta_1\}} \cdots \sum_{\{\theta_n\}} p(\boldsymbol{\theta}, D) = p(D) \quad \leftarrow \text{Law of total probability}$$

- The normalization constant is the probability of the data D and measures how good our probabilistic model explains the observed training set D (*model evidence*)!



$$p(S_1, S_2, P_1, P_2, Y)$$



$$p(S_1, S_2, P_1, P_2, Y = 1)$$

Normalization Constant via Message Passing

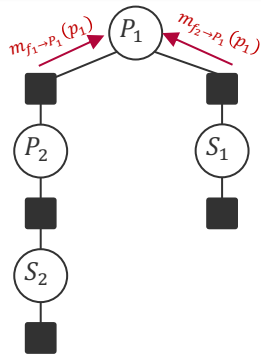
- The non-normalized marginal $p_{X_j}(\cdot)$ is defined as

$$p_{X_j}(x_j) = \sum_{\{x_1\}} \cdots \sum_{\{x_{j-1}\}} \sum_{\{x_{j+1}\}} \cdots \sum_{\{x_n\}} f_1(\mathbf{x}_{\text{ne}(f_1)}) \cdot f_2(\mathbf{x}_{\text{ne}(f_2)}) \cdots f_m(\mathbf{x}_{\text{ne}(f_m)})$$

$$\prod_{i \in \text{ne}(X_j)} m_{f_i \rightarrow X_j}(x_j)$$

Variable node root

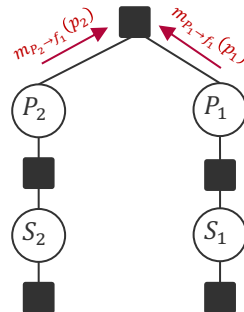
$$Z = \sum_{\{x_j\}} \prod_{i \in \text{ne}(X_j)} m_{f_i \rightarrow X_j}(x_j)$$



$$Z = \sum_{\{x_j\}} p_{X_j}(x_j)$$

Factor node root

$$Z = \sum_{\{\mathbf{x}_{\text{ne}(f_i)}\}} f(\mathbf{x}_{\text{ne}(f_i)}) \prod_{k \in \text{ne}(f_i)} m_{X_k \rightarrow f_i}(x_k)$$



$$m_{f_i \rightarrow X_j}(x_j) \cdot m_{X_j \rightarrow f_i}(x_j)$$

$$\sum_{\{\mathbf{x}_{\text{ne}(f_i) \setminus \{j\}}\}} f(\mathbf{x}_{\text{ne}(f_i)}) \prod_{k \in \text{ne}(f_i) \setminus \{j\}} m_{X_k \rightarrow f_i}(x_k)$$

**Advanced Probabilistic
Machine Learning**

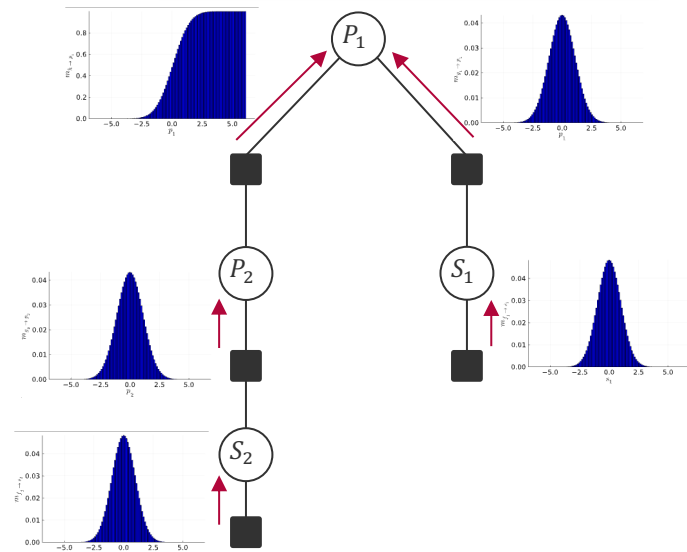
Unit 5 – Approximate Inference:
Expectation Propagation

Normalization Constant via Message Passing: Example

$$\mathcal{N}(s_1; \mu_1, \sigma_1^2) \cdot \mathcal{N}(s_2; \mu_2, \sigma_2^2) \cdot \mathcal{N}(p_1; s_1, \beta^2) \cdot \mathcal{N}(p_2; s_2, \beta^2) \cdot \mathbb{I}(y(p_1 - p_2) > 0)$$

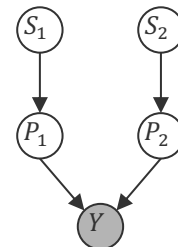
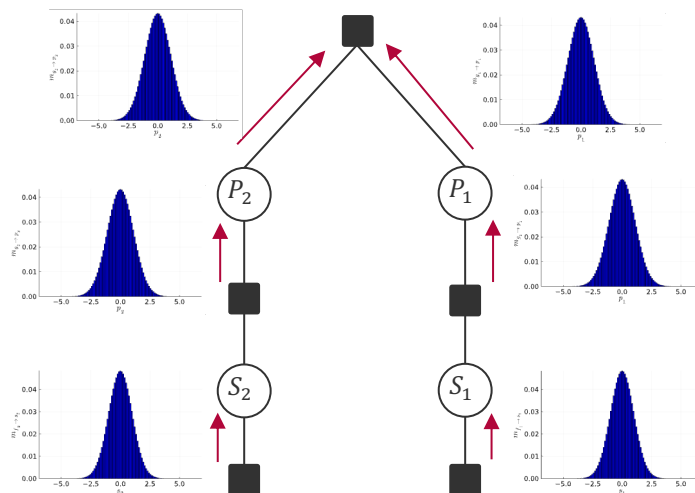
Variable node root

$$Z = \sum_{\{x_j\}} \prod_{i \in \text{ne}(X_j)} m_{f_i \rightarrow X_j}(x_j)$$



Factor node root

$$Z = \sum_{\{x_{\text{ne}(f_i)}\}} f(x_{\text{ne}(f_i)}) \prod_{k \in \text{ne}(f_i)} m_{X_k \rightarrow f_i}(x_k)$$



Advanced Probabilistic Machine Learning

Unit 5 – Approximate Inference:
Expectation Propagation

Normalization Constant via Approximate Message Passing

- **Challenge:** Computing the normalization constant Z requires
 1. Tracking the normalization constants for all messages
 2. Choosing a variable X_j or factor f_i as a root of the factor tree
- **Observation:** Tracking the normalization constant for all messages is not always possible when approximating messages!
- **Theorem:** Given a factor tree with renormalized messages $\tilde{m}_{X_j \rightarrow f_i}(\cdot) = \beta_{j,i} \cdot m_{X_j \rightarrow f_i}(\cdot)$ and $\tilde{m}_{f_i \rightarrow X_j}(\cdot) = \alpha_{i,j} \cdot m_{f_i \rightarrow X_j}(\cdot)$ the normalization constant Z is

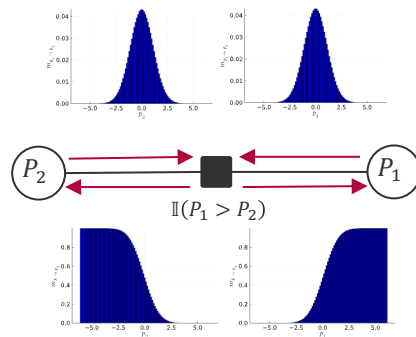
$$Z = \left(\prod_{i=1}^m Z_{f_i} \right) \cdot \left(\prod_{j=1}^n Z_{X_j} \right)$$

Factor Normalization

Variable Normalization

$$Z_{f_i} = \frac{\sum_{\{x_{\text{ne}(f_i)}\}} f(x_{\text{ne}(f_i)}) \prod_{k \in \text{ne}(f_i)} \tilde{m}_{X_k \rightarrow f_i}(x_k)}{\sum_{\{x_{\text{ne}(f_i)}\}} \prod_{k \in \text{ne}(f_i)} \tilde{m}_{f_i \rightarrow X_k}(x_k) \cdot \tilde{m}_{X_k \rightarrow f_i}(x_k)}$$

$$Z_{X_j} = \sum_{\{x_j\}} \prod_{i \in \text{ne}(X_j)} \tilde{m}_{f_i \rightarrow X_j}(x_j)$$



Advanced Probabilistic Machine Learning

Unit 5 – Approximate Inference:
Expectation Propagation

Proof of Normalization Theorem

- Simplifying the variable normalization Z_{X_j} and factor normalization Z_{f_i}

Variable normalization

$$\begin{aligned}
 Z_{X_j} &= \sum_{\{x_j\}} \prod_{i \in \text{ne}(X_j)} \tilde{m}_{f_i \rightarrow X_j}(x_j) \quad \leftarrow \alpha_{i,j} \cdot m_{f_i \rightarrow X_j}(x_j) \\
 &= \left(\prod_{i \in \text{ne}(X_j)} \alpha_{i,j} \right) \cdot \left(\sum_{\{x_j\}} \prod_{i \in \text{ne}(X_j)} m_{f_i \rightarrow X_j}(x_j) \right) \quad \leftarrow Z \\
 &= \left(\prod_{i \in \text{ne}(X_j)} \alpha_{i,j} \right) \cdot Z
 \end{aligned}$$

Factor normalization

$$\begin{aligned}
 Z_{f_i} &= \frac{\sum_{\{x_{\text{ne}(f_i)}\}} f(x_{\text{ne}(f_i)}) \prod_{k \in \text{ne}(f_i)} \tilde{m}_{X_k \rightarrow f_i}(x_k) \quad \leftarrow \beta_{k,i} \cdot m_{X_k \rightarrow f_i}(x_k)}{\sum_{\{x_{\text{ne}(f_i)}\}} \prod_{k \in \text{ne}(f_i)} \tilde{m}_{f_i \rightarrow X_k}(x_k) \cdot \tilde{m}_{X_k \rightarrow f_i}(x_k)} \\
 &= \frac{\cancel{(\prod_{k \in \text{ne}(f_i)} \beta_{k,i})} \cdot \left(\sum_{\{x_{\text{ne}(f_i)}\}} f(x_{\text{ne}(f_i)}) \prod_{k \in \text{ne}(f_i)} m_{X_k \rightarrow f_i}(x_k) \right) \quad \leftarrow Z}{\cancel{(\prod_{k \in \text{ne}(f_i)} \beta_{k,i})} \cdot \left(\prod_{k \in \text{ne}(f_i)} \alpha_{i,k} \right) \cdot \left(\sum_{\{x_{\text{ne}(f_i)}\}} \prod_{k \in \text{ne}(f_i)} m_{f_i \rightarrow X_k}(x_k) \cdot m_{X_k \rightarrow f_i}(x_k) \right)} \\
 &= \frac{Z}{\left(\prod_{k \in \text{ne}(f_i)} \alpha_{i,k} \right) \cdot Z^{|\text{ne}(f_i)|}}
 \end{aligned}$$

$\prod_{k \in \text{ne}(f_i)} \sum_{\{x_k\}} m_{f_i \rightarrow X_k}(x_k) \cdot m_{X_k \rightarrow f_i}(x_k)$

$$\left(\prod_{j=1}^n Z_{X_j} \right) \cdot \left(\prod_{i=1}^m Z_{f_i} \right) = \left(\prod_{j=1}^n \prod_{i \in \text{ne}(X_j)} \alpha_{i,j} \right) \cdot Z^n \cdot \frac{Z^m}{\left(\prod_{i=1}^m \prod_{k \in \text{ne}(f_i)} \alpha_{i,k} \right) \cdot \prod_{i=1}^m Z^{|\text{ne}(f_i)|}}$$

identical

Z^{n+m-1} because the factor graph is a tree with $n + m$ nodes!

Overview

1. Distance Measures for Distributions
2. Approximating Marginals: Expectation Propagation
3. Approximating Normalization Constants
4. **Application: Bayesian Ranking**

**Advanced Probabilistic
Machine Learning**

*Unit 5 – Approximate Inference:
Expectation Propagation*

The Skill Rating Problem

■ Given:

- **Match outcomes:** Orderings among k teams consisting of $n_1, n_2, \dots, n_k$ players.

Team		Score				
1st	Red Team	50				
2nd	Blue Team	40				

	Level	Gamertag	Avg. Life	Best Spree	Score
1st	10	BlueBot	00:00:49	6	15
1st	7	SniperEye	00:00:41	4	14
1st	9	ProThePirate	00:01:07	3	13
1st	10	dazdemon	00:00:59	3	8
2nd	10	WastedHarry	00:00:41	4	17
2nd	3	Ascla	00:00:37	2	10
2nd	9	Antidote4Losing	00:00:41	2	9
2nd	12	Blackknight9	00:00:48	3	4

	Level	Gamertag	Avg. Life	Best Spree	Score
1st	N/A	SniperEye	N/A	N/A	25
2nd	N/A	xxXHALOxxX	N/A	N/A	24
3rd	N/A	AjaySandhu	N/A	N/A	15
3rd	N/A	AjaySandhu(G)	N/A	N/A	15
5th	N/A	Robert115	N/A	N/A	11
5th	N/A	TurboNegro84(G)	N/A	N/A	11
7th	N/A	TurboNegro84	N/A	N/A	5
8th	N/A	SniperEye(G)	N/A	N/A	1

■ Questions:

1. Skill s_i for each player such that $s_i > s_j \Leftrightarrow P(\text{Player } i \text{ wins}) > P(\text{Player } j \text{ wins})$
2. Global ranking among all players
3. Fair matches between teams of players

**Advanced Probabilistic
Machine Learning**

*Unit 5 – Approximate Inference:
Expectation Propagation*

Two-Player Match Outcome Model

- **Simple Two-Player Games:** Our data is the identity i and j of the two players and the outcome $Y \in \{-1, +1\}$ of a match between them.

- **Bradley-Terry Model (1952):** Model of a win of player i given skills s_i and s_j is

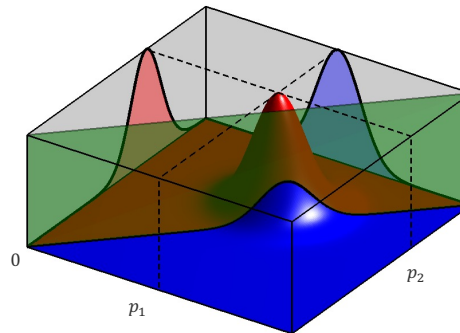
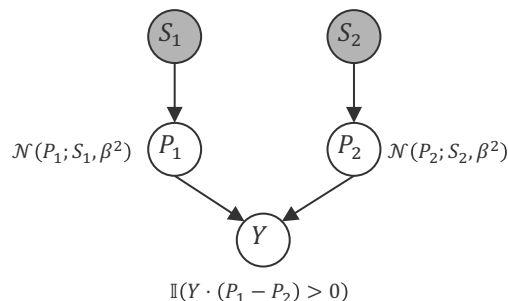
$$P(Y = 1 | s_i, s_j) = \frac{\exp(s_i)}{\exp(s_i) + \exp(s_j)} = \frac{\exp(s_i - s_j)}{1 + \exp(s_i - s_j)}$$

Logistic sigmoid in skill difference

- **Thurstone Case V Model (1927):** Model of a win given skills s_i and s_j is

$$P(Y = 1 | s_i, s_j) = \int_0^{\infty} \mathcal{N}(t; s_i - s_j, 2\beta^2) dt$$

Probit sigmoid in skill difference



Ralph A. Bradley
(1923 – 2001)



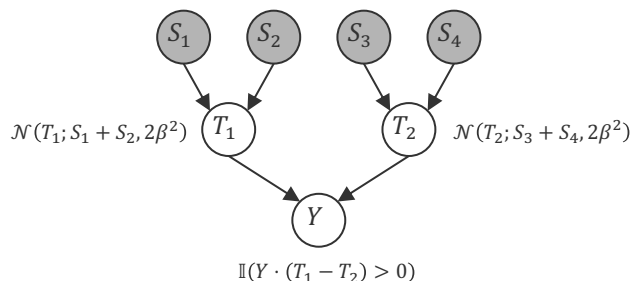
Louis Leon Thurstone
(1887 – 1955)

**Advanced Probabilistic
Machine Learning**

Unit 5 – Approximate Inference:
Expectation Propagation

Two-Team Match Outcome Model

- **Team Assumption:** Performance of a team is the sum of the performances of its players



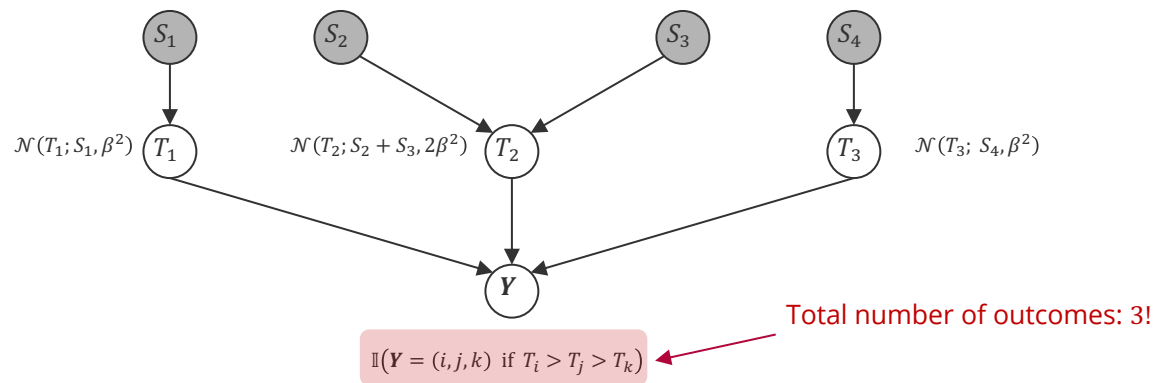
- **Pro:** Games where the team scores are additive (e.g., kill count in first-person shooter)
- **Con:** Games where the outcome is determined by a single player (e.g., fastest car in a race)
- **Observation:** Match outcomes correlate the skills of players
 - **Same Team:** Anti-correlated
 - **Opposite Teams:** Correlated

**Advanced Probabilistic
Machine Learning**

*Unit 5 – Approximate Inference:
Expectation Propagation*

Multi-Team Match Outcome Model

- **Possible Outcomes:** Permutations $\mathbf{Y} \in \{1,2,3\}^3$ of teams



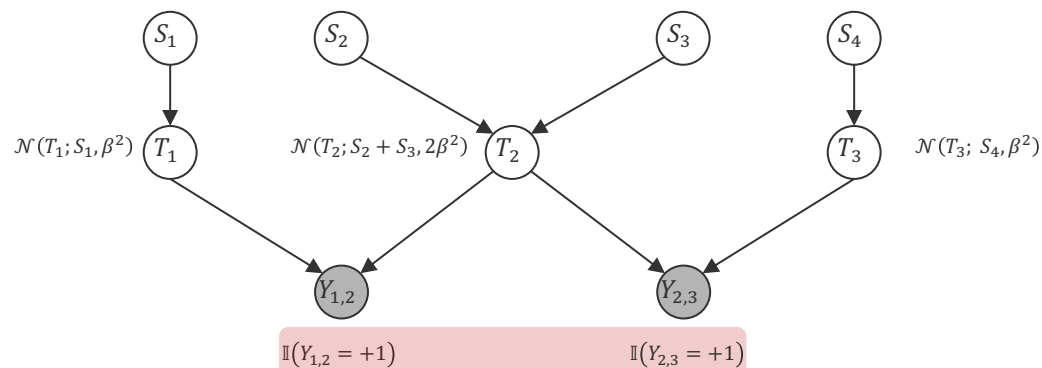
- **Easy to sample** for given skills but computationally difficult to "invert"!

**Advanced Probabilistic
Machine Learning**

Unit 5 – Approximate Inference:
Expectation Propagation

From Match Outcomes to Pairwise Rankings

- **Learning:** In the ranking setting, we observe multi-team match outcomes and want to infer the skills of all single players!
- **Idea:** Leverage the transitivity of the real line of latent scores!



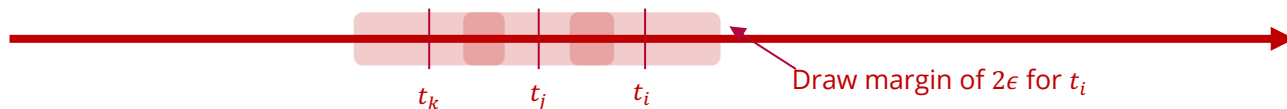
Advanced Probabilistic Machine Learning

Unit 5 – Approximate Inference:
Expectation Propagation

By transitivity, this implies $T_1 > T_3$

Modelling Draws and Partial Play

- **Draw Model:** Instead of $t_i > t_j$ for the winning team, we have three outcomes ($\epsilon > 0$)
 - **Team i wins:** $t_i > t_j + \epsilon \Leftrightarrow t_i - t_j > \epsilon$
 - **Team j wins:** $t_j > t_i + \epsilon \Leftrightarrow t_j - t_i > \epsilon$
 - **Teams draw:** $t_i \leq t_j + \epsilon$ and $t_j \leq t_i + \epsilon \Leftrightarrow |t_i - t_j| \leq \epsilon$
 - Pairwise draws in a chain **do not** model the actual event that all pairwise team performances are at most ϵ away from each other!



- **Partial Play:** If a player i only participates for a fraction $\alpha_i \in [0,1]$ of the time in the match, then we model this assuming a linear contribution to the team skill by

$$T \sim \mathcal{N}(\cdot; \alpha_1 s_1 + \alpha_2 s_2, (\alpha_1^2 + \alpha_2^2) \beta^2)$$

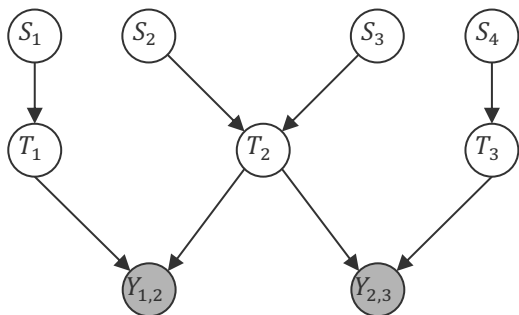
- This only works if the fraction α_i is truly independent of the (predicted) match outcome!

Advanced Probabilistic
Machine Learning

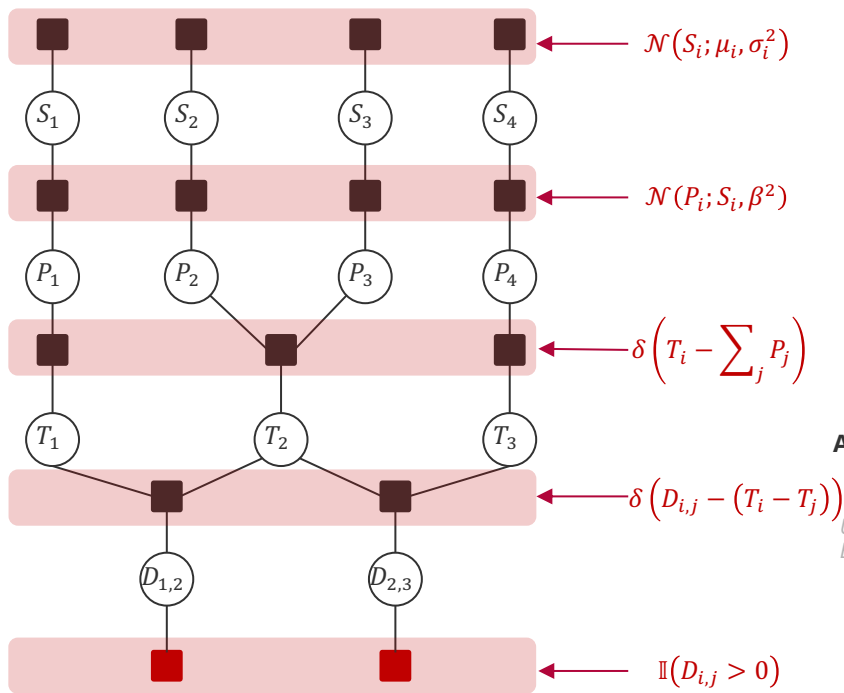
Unit 5 – Approximate Inference:
Expectation Propagation

TrueSkill Factor Tree

Bayesian Network



Factor Graph

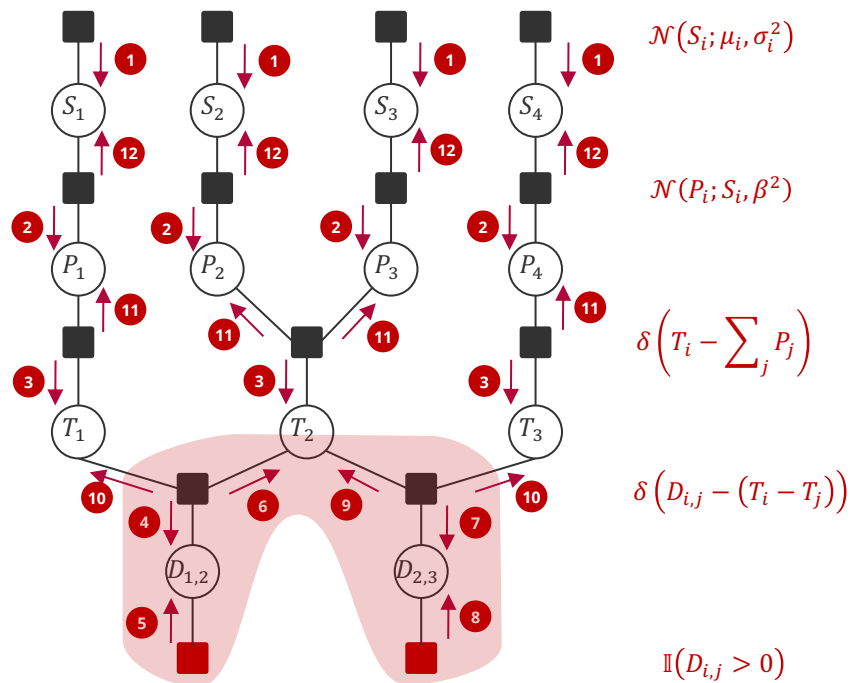


**Advanced Probabilistic
Machine Learning**

Unit 5 – Approximate Inference:
Expectation Propagation

(Approximate) Message Passing in TrueSkill Factor Trees

TrueSkill Factor Graph



■ Four Phases

1. Pass prior messages (1)
2. Pass messages *down* to the team performances (2 to 3)
3. Iterate the approximate messages on the pairwise team differences (4 to 9)
4. Pass messages back from *up* from team performances to player skill (10 – 12)

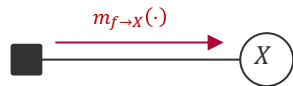
■ Since this is a *tree*, the algorithm is guaranteed to converge!

Advanced Probabilistic Machine Learning

Unit 5 – Approximate Inference:
Expectation Propagation

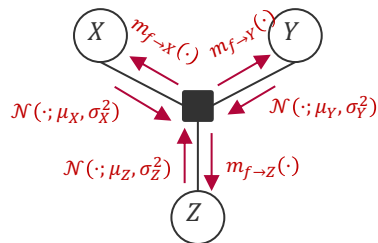
Message Update Equations

Gaussian Factor $\mathcal{N}(X; \mu, \sigma^2)$



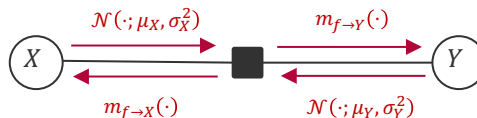
$$m_{f \to X}(x) = \mathcal{N}(x; \mu, \sigma^2)$$

Weighted Sum Factor $\delta(Z - (aX + bY))$



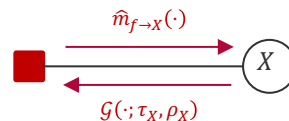
$$\begin{aligned} m_{f \to Z}(z) &= \mathcal{N}(z; a\mu_X + b\mu_Y, a^2\sigma_X^2 + b^2\sigma_Y^2) \\ m_{f \to Y}(y) &= \mathcal{N}(y; (\mu_Z - a\mu_X)/b, (\sigma_Z^2 + a^2\sigma_X^2)/b^2) \\ m_{f \to X}(x) &= \mathcal{N}(x; (\mu_Z - b\mu_Y)/a, (\sigma_Z^2 + b^2\sigma_Y^2)/a^2) \end{aligned}$$

Gaussian Mean Factor $\mathcal{N}(Y; X, \beta^2)$



$$\begin{aligned} m_{f \to Y}(y) &= \mathcal{N}(y; \mu_X, \sigma_X^2 + \beta^2) \\ m_{f \to X}(x) &= \mathcal{N}(x; \mu_Y, \sigma_Y^2 + \beta^2) \end{aligned}$$

Between Factor $\mathbb{I}(l \leq X < u)$



$$\hat{m}_{f \to X}(x) = \mathcal{G}\left(x; \sqrt{\rho_X} \cdot \frac{V}{1 - W} + \tau_X \cdot \frac{W}{1 - W}, \rho_X \cdot \frac{W}{1 - W}\right)$$

Correction functions

$$V := v_{l, \sqrt{\rho_X}, u, \sqrt{\rho_X}}\left(\frac{\tau_X}{\sqrt{\rho_X}}\right)$$

$$W := w_{l, \sqrt{\rho_X}, u, \sqrt{\rho_X}}\left(\frac{\tau_X}{\sqrt{\rho_X}}\right)$$

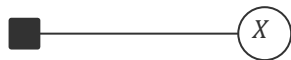
of doubly-truncated Gaussians

Advanced Probabilistic Machine Learning

Unit 5 – Approximate Inference:
Expectation Propagation

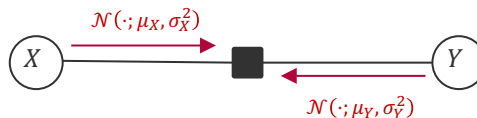
Factor Normalizations

Gaussian Factor $\mathcal{N}(X; \mu, \sigma^2)$



$$Z_f = 1$$

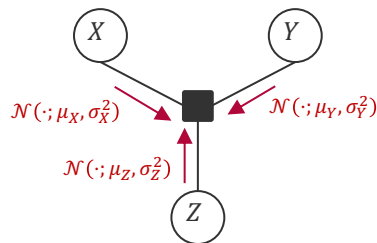
Gaussian Mean Factor $\mathcal{N}(Y; X, \beta^2)$



$$Z_f = \frac{1}{\mathcal{N}(\mu_X; \mu_Y, \sigma_X^2 + \sigma_Y^2 + \beta^2)}$$

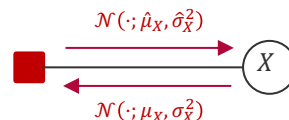
$$Z_{f_i} = \frac{\sum_{\{x_{ne(f_i)}\}} f_i(x_{ne(f_i)}) \prod_{j \in \text{ne}(f_i)} \hat{m}_{X_j \rightarrow f_i}(x_j)}{\sum_{\{x_{ne(f_i)}\}} \prod_{j \in \text{ne}(f_i)} \hat{m}_{f_i \rightarrow X_j}(x_j) \cdot \hat{m}_{X_j \rightarrow f_i}(x_j)}$$

Weighted Sum Factor $\delta(Z - (aX + bY))$



$$Z_f = \frac{1}{|a| \cdot |b| \cdot [\mathcal{N}(\mu_Z; a\mu_X + b\mu_Y, \sigma_Z^2 + a^2\sigma_X^2 + b^2\sigma_Y^2)]^2}$$

Between Factor $\mathbb{I}(l \leq X < u)$



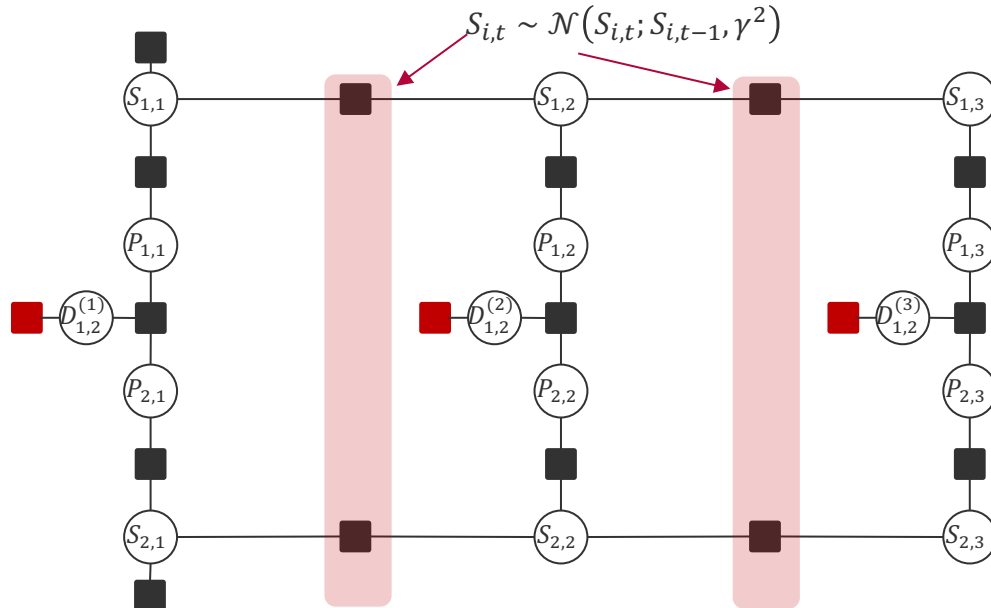
$$\Phi(z) := \int_{-\infty}^z \mathcal{N}(x; 0, 1) dx$$

$$Z_f = \frac{\Phi\left(\frac{u - \mu_X}{\sigma_X}\right) - \Phi\left(\frac{l - \mu_X}{\sigma_X}\right)}{\mathcal{N}(\hat{\mu}_X; \mu_X, \sigma_X^2 + \hat{\sigma}_X^2)}$$

Advanced Probabilistic Machine Learning

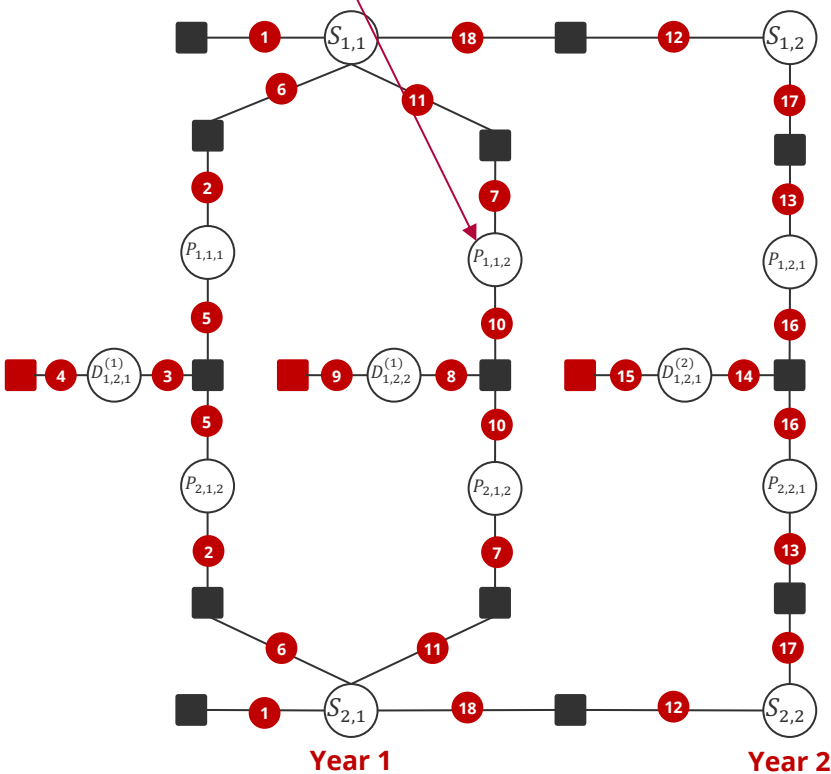
Unit 5 – Approximate Inference:
Expectation Propagation

- **Dynamics:** In reality, skills of players evolve over time and are not stationary
 - **Idea:** Since we do not know which direction the skills evolve, assume that the skill of player i at time t depends on the skill of the same player at time $t - 1$ via



TrueSkill Through Time: Message Schedule

Performance of player 1 in year 1
in second match



■ Four Phases

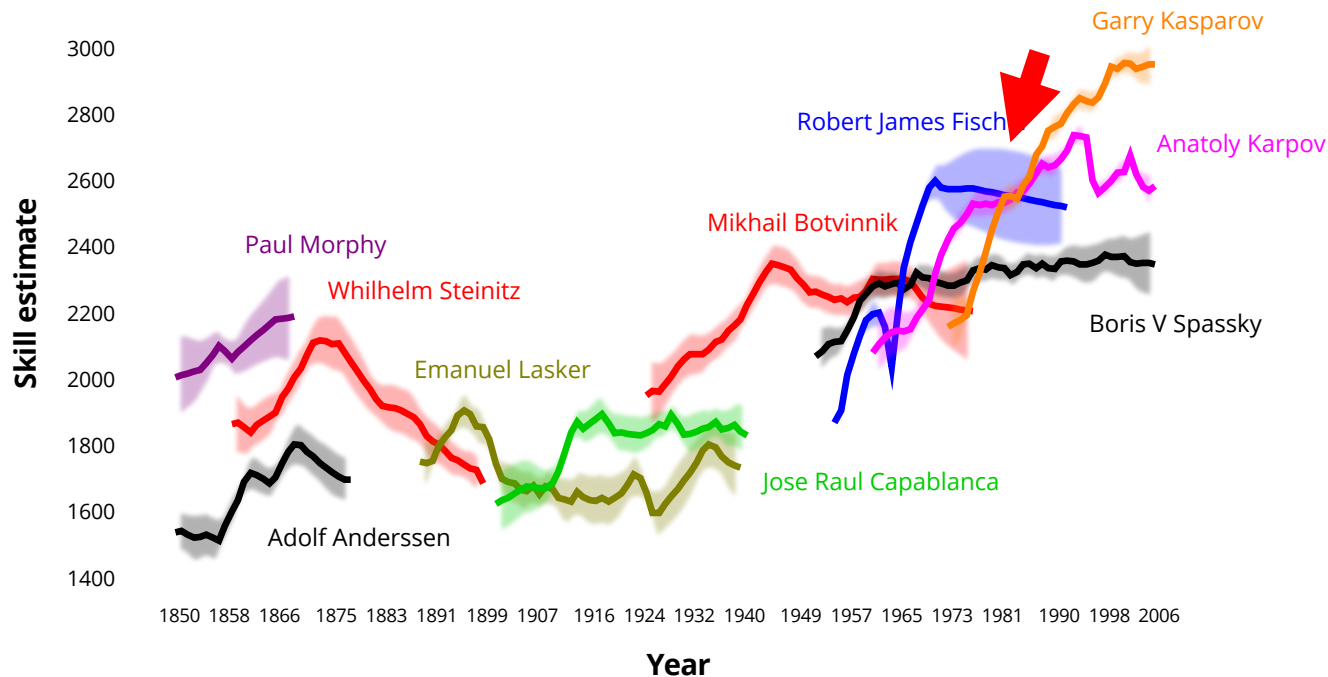
1. **Prior (1):** Send prior messages to each skill variable for the first year of a player
2. **Annual Matches (2-11):** Loop over all (2-player) matches in a year until the skill marginals for all active player in that year does not change (much) anymore
3. **Forward Dynamics (12):** Send skill dynamics messages forward in time from t to $t + 1$ and keep running phase 2. (13 – 17).
4. **Backward Dynamics (18):** Send skill dynamics messages backward in time from year $t + 1$ to t and keep running step 2. (2-11)

Advanced Probabilistic Machine Learning

*Unit 5 – Approximate Inference:
Expectation Propagation*

- Stop when no variable in the outer loop changes much anymore.

TrueSkill-Through-Time: Chess Players



History of Chess
3.5M match outcomes
20 million variables
40 million factors

**Advanced Probabilistic
Machine Learning**

*Unit 5 – Approximate Inference:
Expectation Propagation*

■ Distance Measures for Distributions

- α -divergences are a general class of distance measures between distributions
- For $\alpha = 1$, the α -divergence becomes the Kullback-Leibler divergence where the minimizer for Gaussian approximating distributions matches mean and variance
- Minimizers of α -divergences range from mode-seeking to support-seeking

■ Approximate Message Passing and Expectation Propagation

- Approximations will always be done on the marginals, **not** the messages
- When the Kullback-Leibler divergence is used as distance, all moments get preserved
- In case of doubly-truncated Gaussians, the moments are closed form

■ Approximating Normalization Constants

- If factor graphs represent joint probabilities of data and parameters with data variables only, the normalization constant equals the probability of data
- If messages can be explicitly represented and normalized, efficient updates can be done starting at any factor or variable
- There is a general distributed algorithm for approximating the normalization constant

See you next week!